

INFORME TÉCNICO

Módulo Clientes y arquitectura de la SPA PharmaSoft

Implementación revisada desde el código fuente del repositorio pharma-frontend. Las referencias entre paréntesis apuntan a archivo y líneas actuales.

B1. Estructura del proyecto

```
src/app/
├── core/ configuración y modelos compartidos
├── config/menu.ts
├──
├── models/{error-response,pagina-response}.ts
├── utils/http-error.ts
├── shared/pages/no-encontrado/
├── layout/{header,main-layout,sidebar}/
├── features/
├── inicio/
├── categorias/{models,services,pages}/ +
  categorias.routes.ts
├── clientes/{models,services,pages}/ +
  clientes.routes.ts
├── app.routes.ts
└── app.config.ts
```

core: tipos y utilidades transversales. shared: vistas reutilizables ajenas a un módulo. layout: estructura persistente de navegación. features: funcionalidades agrupadas por dominio; cada módulo contiene modelos, servicio, páginas y rutas.

B2. Mapa de rutas

Ruta	Componente	Padre	Carga	Título
/	Redirige a /inicio	MainLayout	redirectTo	-
/inicio	Inicio	MainLayout	loadComponent	Inicio
/categorias	CategoriaList	MainLayout	loadChildren	Categorías
/categorias/nuevo	CategoriaForm	MainLayout	loadChildren	Nueva categoría
/categorias/:id/editar	CategoriaForm	MainLayout	loadChildren	Editar categoría
/clientes	ClienteList	MainLayout	loadChildren	Clientes
/clientes/nuevo	ClienteForm	MainLayout	loadChildren	Nuevo cliente
/clientes/:id/editar	ClienteForm	MainLayout	loadChildren	Editar cliente
/**	NoEncontrado	-	loadComponent	Página no encontrada

Fuente: app.routes.ts, líneas 4-29; rutas de Clientes en clientes.routes.ts, líneas 5-9 y Categorías en categorias.routes.ts, líneas 5-9. Las rutas hijas se resuelven bajo MainLayout.

B3. Responsabilidades

La separación evita que la vista implemente detalles HTTP y que el servicio conozca decisiones de presentación.

Pieza / tipo	Responsabilidad	Qué no hace
CategoriaService / servicio	CRUD HTTP de categorías, Observables tipados.	No renderiza mensajes ni decide navegación.
CategoriaList / componente	Carga, filtra y presenta categorías; confirma eliminación.	No construye peticiones con HttpClient.
CategoriaForm / componente	Valida y envía alta/edición; navega al terminar.	No conoce detalles de transporte HTTP.
ClienteService / servicio	CRUD de clientes y parámetros de página/orden; encapsula endpoint.	No presenta errores ni gestiona formularios.
ClienteList / componente	Estado de página, búsqueda local, orden, carga y baja lógica.	No contiene HttpClient ni construye URL.
ClienteForm / componente	Valida datos, convierte opcionales vacíos a null y registra/edita.	No decide reglas de unicidad; backend responde conflictos.
Header / componente layout	Marca, navegación superior y evento para alternar menú.	No carga ni cambia la ruta del módulo activo.
Sidebar / componente layout	Pinta enlaces configurados en MENU y marca ruta activa.	No consulta clientes ni aplica permisos de servidor.
MainLayout / componente layout	Mantiene header/sidebar y aloja la vista hija en RouterOutlet.	No implementa lógica de negocio de features.

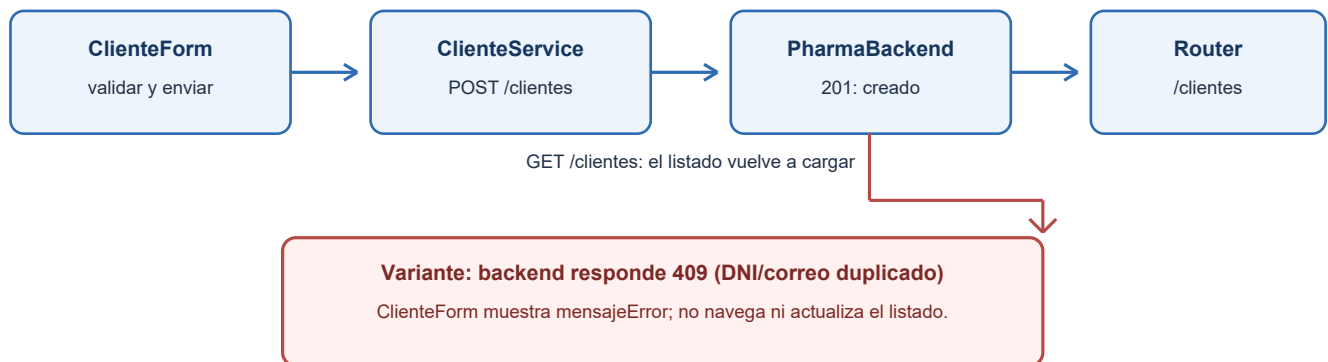
Evidencia del código

CategoriaService concentra llamadas HTTP (categoria-service.ts: 6-29); ClienteService hace lo propio y tipa la respuesta paginada o el arreglo que entrega el backend actual (cliente-service.ts: 8-25). ClienteList administra signals y filtro derivado (cliente-list.ts: 10-24), mientras ClienteForm define validadores (cliente-form.ts: 19-27) y transforma telefono/direccion vacíos a null (línea 44).

El flujo de errores usa mensajeError compartido: los componentes consumen el campo message de ErrorResponse y muestran un fallback por status (core/utls/http-error.ts: 4-15).

B4. Flujo de “Registrar cliente”

La navegación de éxito vuelve al listado; el listado solicita los datos y los almacena en signals. Si hay conflicto, la navegación no ocurre.



Referencias: `ClienteForm.guardar` (cliente-form.ts: 40-49), `ClienteService.crear` (cliente-service.ts: 23) y `ClienteList.cargar` (cliente-list.ts: 27-50). El 409 se convierte en texto con `mensajeError`; el formulario lo presenta en el bloque `role="alert"`.

B5. Preguntas de revisión

1. ¿Cuántos archivos existentes se modificaron? Dos: `src/app/app.routes.ts` (líneas 19-22) y `src/app/core/config/menu.ts` (línea 10). El resto del feature se agregó en archivos nuevos; esto muestra que el módulo puede incorporarse con cambios acotados a composición y navegación.
2. ¿Por qué `ClienteList` no usa `HttpClient` directamente? Porque `ClienteService` concentra URL, parámetros y operaciones. Si cambia el endpoint, se ajusta el servicio sin propagar ese detalle a la vista; la inyección del servicio mantiene la responsabilidad de presentación separada. (cliente-list.ts: 11, cliente-service.ts: 8-25).
3. ¿Qué ventaja tiene `loadChildren`? Las rutas de Clientes se cargan de forma diferida y su código no se incluye en el chunk inicial. La salida de `npm run build` generó el chunk `clientes-routes` (18.39 kB, compilación revisada el 01-10-2026). Esto reduce el código inicial descargado; Angular obtiene el chunk cuando se navega a Clientes. (app.routes.ts: 19-22).

Dato de compilación capturado de la salida revisada: chunk-DwA2e2ss.js | clientes-routes | 18.39 kB | 5.09 kB transfer.

B5. Preguntas de revisión (continuación)

4. ¿Por qué el encabezado y el sidebar no se vuelven a dibujar al cambiar de módulo? MainLayout es el componente padre de las rutas funcionales y declara header, sidebar y RouterOutlet. La navegación cambia el componente hijo del outlet mientras se conserva la instancia del layout (app.routes.ts: 5-8; main-layout.html: 1-6).

5. Si ciertos usuarios no deben ver Clientes, ¿dónde se cambia? Para ocultar el enlace de navegación, en MENU (core/config/menu.ts: 7-11), que Sidebar itera para construir los enlaces (sidebar.ts: 3, 12-13). Ese cambio solo oculta el enlace: para controlar acceso real se debe añadir un guard de ruta y autorización equivalente en el backend.

6. ¿Qué código está duplicado y cómo podría reutilizarse? Los módulos repiten patrones de tabla/estados, formulario reactivo y manejo de errores. Una opción es extraer componentes visuales comunes (búsqueda, estado vacío, paginador) y utilidades compartidas; los servicios y modelos seguirían siendo específicos de cada dominio.

7. ¿Por qué la búsqueda solo filtra la página actual y qué cambiaría? El computed solo recorre el array clientes ya cargado en el componente (cliente-list.ts: 21-24). Para buscar en todos los registros, el backend debe aceptar criterios como dni y texto de nombres/apellidos antes de paginar; ClienteService debería añadir esos valores a HttpParams y el componente reiniciar pagina a 0 al cambiar la búsqueda.

Compatibilidad de respuesta observada en el código

La especificación de la actividad describe una respuesta PaginaResponse. Durante la integración, el endpoint local respondió un arreglo JSON; por eso ClienteService acepta ambos tipos y ClienteList pagina/ordena localmente cuando recibe un arreglo (cliente-service.ts: 13-19; cliente-list.ts: 30-48). Cuando recibe el objeto paginado, utiliza su contenido y metadatos directamente.

Alcance de las evidencias

Compilación: npm run build finalizó correctamente y produjo un chunk separado para clientes-routes. No se ejecutaron los once casos de interfaz indicados en A7; este informe no afirma que se hayan validado manualmente. Las capturas de Network de esos casos deben añadirse después de ejecutar la matriz de pruebas en el navegador.

Conclusión

Clientes reutiliza la estructura por feature: un cambio de rutas, una entrada en el menú y archivos propios para el dominio. El layout y Categorías conservan sus responsabilidades. La coexistencia de respuestas paginadas y arreglo queda explícita en el código para ajustarse al backend disponible.